

Primeiro Contato com Jetpack Compose

Desenvolvimento de Software para Dispositivos Móveis

Prof. Me. Sergio Souza Novak — UniRV

13 de agosto de 2026



O que é Jetpack Compose?

- É o framework moderno do Android para criação de interfaces de usuário.
- Usa a linguagem Kotlin de forma nativa.
- Substitui o sistema tradicional de Views (XML).

Em resumo

Compose transforma o modo como criamos interfaces Android.

- Menos código.
- Mais rápido (visualmente).
- Código mais legível.

Não é uma biblioteca, é um sistema de UI Declarativo

- Em vez de dizer **como** desenhar cada componente (imperativo),

O que é Jetpack Compose?

- É o framework moderno do Android para criação de interfaces de usuário.
- Usa a linguagem Kotlin de forma nativa.
- Substitui o sistema tradicional de Views (XML).

Em resumo

Compose transforma o modo como criamos interfaces Android.

- Menos código.
- Mais rápido (visualmente).
- Código mais legível.

Não é uma biblioteca, é um sistema de UI Declarativo

- Em vez de dizer **como** desenhar cada componente (imperativo),
- Você apenas **descreve** como a tela deve ser para cada estado

O que é Jetpack Compose?

- É o framework moderno do Android para criação de interfaces de usuário.
- Usa a linguagem Kotlin de forma nativa.
- Substitui o sistema tradicional de Views (XML).

Em resumo

Compose transforma o modo como criamos interfaces Android.

- Menos código.
- Mais rápido (visualmente).
- Código mais legível.

Não é uma biblioteca, é um sistema de UI Declarativo

- Em vez de dizer **como** desenhar cada componente (imperativo),
- Você apenas **descreve** como a tela deve ser para cada estado
- O Compose cuida de desenhar e atualizar a tela automaticamente.

UI Declarativa vs UI Imperativa

Pensamento moderno

UI Imperativa (ViewSystem)

- Você atualiza elementos da tela manualmente.
- Exemplo: `TextView.setText("Olá")`
- Se a tela tiver 3 estados, você escreve código para cada estado.

UI Declarativa (Compose)

- Você descreve a tela para cada estado.
- Exemplo: Função `MeuTexto(texto="Olá")`
- Se a tela tiver 3 estados, você escreve 3 funções (ou uma lógica).

Criando o Projeto MagicBall

- Abra o Android Studio.
- Crie um novo projeto **Empty Activity** (com Jetpack Compose).
- **Name:** MagicBall
- **Package name:** com.sergiosouzanovak.magicball
- **Save location:** /AndroidStudioProjects/MagicBall
- **Language:** Kotlin

MainActivity e Tema

A porta de entrada do nosso app. Configura o edge-to-edge e o conteúdo com o tema.

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            MagicBallTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    App(modifier = Modifier.padding(innerPadding))  
                }  
            }  
        }  
    }  
}
```

O Modelo de Dados: Message

Vamos criar uma estrutura simples para armazenar a mensagem que será exibida.

```
data class Message(  
    val autor:String,  
    val body:String  
)
```

O Componente MessageCard - Parte 1

Vamos construir o cartão da mensagem. Usaremos Row e Column.

```
@Composable
fun MessageCard(message: Message) {
    Row {
        Column {
            Spacer(modifier = Modifier.height(4.dp))
            Image(
                painter = painterResource(R.drawable.img),
                contentDescription = "Contact profile picture",
                modifier = Modifier.size(40.dp).clip(CircleShape)
            )
        }
        Spacer(modifier = Modifier.width(12.dp))
        // Textos virão aqui
    }
}
```

O Componente MessageCard - Parte 2

Adicionando o autor e o corpo da mensagem.

```
@Composable
fun MessageCard(message: Message) {
    Row {
        // ... Imagem (Parte 1)
        Spacer(modifier = Modifier.width(12.dp))

        Column {
            Text(text = message.autor,
                color = MaterialTheme.colorScheme.primary)
            Spacer(modifier = Modifier.width(4.dp))
            Text(text = message.body,
                color = MaterialTheme.colorScheme.secondary)
        }
    }
}
```

Juntando tudo no componente App

O App é onde colocamos os cartões de mensagem.

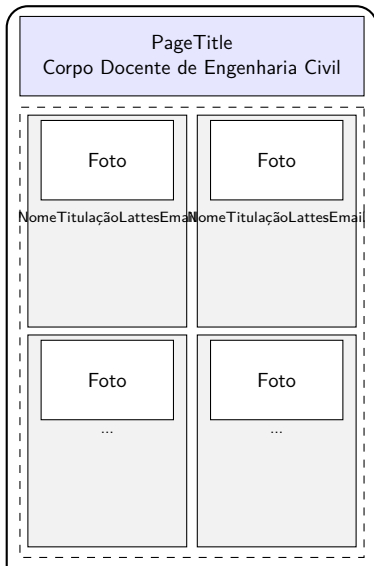
```
@Composable
fun App(modifier: Modifier = Modifier) {
    Column(modifier = modifier) {
        MessageCard(
            Message("Lexi", "corpo da mensagem")
        )
    }
}
```

Pré-visualização (Preview)

Podemos ver o resultado sem rodar o app inteiro, testando temas claro e escuro.

```
@Preview(name = "Light Mode")
@Preview(
    uiMode = Configuration.UI_MODE_NIGHT_YES,
    showBackground = true,
    name = "Dark Mode"
)
@Composable
fun GreetingPreview() {
    MagicBallTheme {
        App()
    }
}
```

Wireframe do App: Corpo Docente



Carregamento de Imagens e Modelo de Dados

A função auxiliar `carregarImagemAsset` e a data class `Docente`.

```
data class Docente(  
    val nome: String, val titulacao: String,  
    val lattes: String, val email: String, val foto: String  
)  
  
@Composable  
fun carregarImagemAsset(nomeArquivo: String): ImageBitmap? {  
    val context = LocalContext.current  
    return try {  
        context.assets.open(nomeArquivo).use { inputStream ->  
            BitmapFactory.decodeStream(inputStream)?.asImageBitmap()  
        }  
    } catch (e: Exception) { null }  
}
```

O Componente PesquisadorCard

Exibe os dados de um docente individualmente.

```
@Composable
fun PesquisadorCard(nome: String, titulacao: String, lattes: String, email: String, foto: String, modifier:
    Modifier = Modifier) {
    val imagem = carregarImagemAsset(foto)
    Card(modifier = modifier.fillMaxWidth().padding(16.dp), colors = CardDefaults.cardColors(containerColor =
        Color(0xFFFF1F4F5))) {
        Column(modifier = Modifier.width(1200.dp).height(300.dp), horizontalAlignment = Alignment.
            CenterHorizontally) {
            imagem?.let { Image(bitmap = it, contentDescription = "Foto", modifier = Modifier.fillMaxWidth().
                height(120.dp), contentScale = ContentScale.Crop) }
            Text(text = nome, fontSize = 12.sp, fontWeight = FontWeight.Bold, color = Color(0xFF222222))
            Text(text = titulacao, fontSize = 12.sp, color = Color(0xFF666666))
            Text(text = lattes, fontSize = 12.sp, color = Color(0xFF007AFF))
            Text(text = email, fontSize = 12.sp, color = Color(0xFF444444))
        }
    }
}
```

Listando os Docentes com Grid

LazyVerticalGrid organiza os cartões em colunas.

```
@Composable
fun ListaPesquisadores(){
    val docentes = listOf(/* instâncias de Docente omitidas por espaço */)
    LazyVerticalGrid(
        columns = GridCells.Fixed(2),
        modifier = Modifier.fillMaxSize(),
        contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        items(docentes) { docente ->
            PesquisadorCard(
                nome = docente.nome,
                titulacao = docente.titulacao,
                lattes = docente.lattes,
                email = docente.email,
                foto = docente.foto
            )
        }
    }
}
```

Cabeçalho e Componente Principal

```
@Composable
fun PageTitle(){
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Row{
            Text(text = "Corpo Docente do curso de")
            Text(text = " Engenharia Civil", color = Color(0xFF064F9F))
        }
        Row{
            Column {
                repeat(2){
                    HorizontalDivider(modifier = Modifier.width(125.dp), thickness = 2.dp, color = Color.Gray)
                    Spacer(modifier = Modifier.height(4.dp))
                }
            }
        }
        Row{ ListaPesquisadores() }
    }
}

@Composable
fun App(modifier: Modifier) {
    Column{ PageTitle() }
}
```